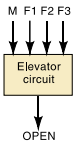
# CPE 204 – Combinational Circuit Applications

1. Design a logic circuit that controls an elevator door in a three-story building. *M* is a logic signal that indicates when the elevator is moving (*M* = 1) or stopped (*M* = 0). *F1*, *F2*, and *F3* are floor indicator signals that are normally LOW, and they go HIGH only when the elevator is positioned at the level of that particular floor. For example, when the elevator is lined up level with the second floor, *F2* = 1 and *F1* = *F3* = 0. The circuit output is the *OPEN* signal, which is normally LOW and will go HIGH when the elevator door is to be opened.

   The truth table for an open circuit is as follows:

a. Because the elevator cannot be lined up with more than one floor at a time, only one of the floor inputs can be HIGH at any given time. This means that all those cases in the truth table where more than one floor input is a 1 are don't-care conditions. We can place an *x* in the *OPEN* output column for those eight cases where more than one *F* input is 1.

b. Looking at the other eight cases, when *M* = 1, the elevator is moving, so *OPEN* must be a 0 because we do not want the elevator door to open. When *M* = 0 (elevator stopped), we want OPEN = 1, provided that one of the floor inputs is 1. When *M* = 0 and all floor inputs are 0, the elevator is stopped but is not properly lined up with any floor, so we want *OPEN* = 0 to keep the door closed.




| M | F1 | F2 | F3 | OPEN |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | X |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | X |
| 0 | 1 | 1 | 0 | X |
| 0 | 1 | 1 | 1 | X |
| 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | X |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | X |
| 1 | 1 | 1 | 0 | X |
| 1 | 1 | 1 | 1 | X |

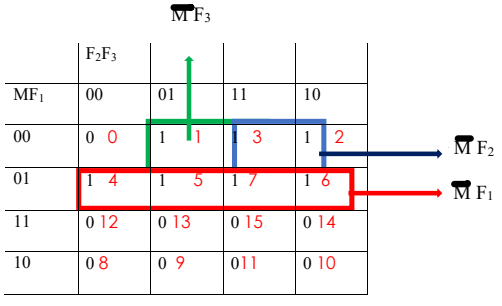
(b)

The truth table is now complete, and we can transfer its information to the K map below. The map has only three 1s, but it has eight don't-cares. By changing four of these don't-care squares to 1s, we can produce quad loopings that contain the original 1s.

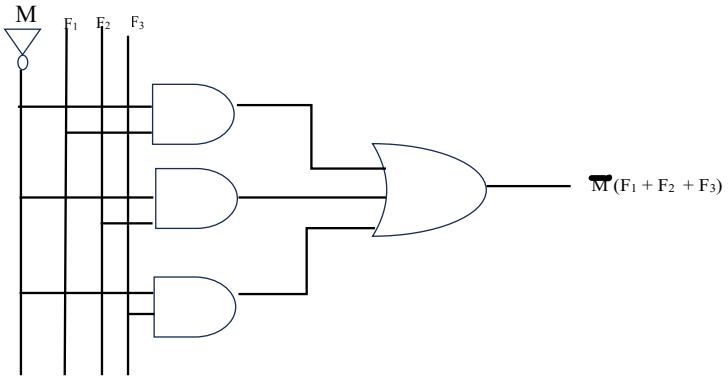
**K-MAP**

| | $F_2F_3$ | | | |
|---|---|---|---|---|
| $MF_1$ | 00 | 01 | 11 | 10 |
| 00 | 0 0 | 1 1 | X 3 | 1 2 |
| 01 | 1 4 | X 5 | X 7 | X 6 |
| 11 | 0 12 | X 13 | X 15 | X 14 |
| 10 | 0 8 | 0 9 | X11 | 0 10 |

**Transformational K-MAP**



$$OPEN = \overline{M}F_1 + \overline{M}F_2 + \overline{M}F_3 = \overline{M}(F_1 + F_2 + F_3)$$




2. An automobile alarm circuit is used to detect certain undesirable conditions. The three switches are used to indicate the status of the door by the driver's seat, the ignition, and the headlights, respectively. Design the logic circuit with these three switches as inputs so that the alarm will be activated whenever either of the following conditions exists:
   i. The headlights are on while the ignition is off.
   ii. The door is open while the ignition is on.

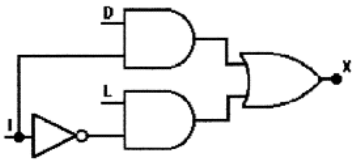
Let the three inputs be D = Door, I = Ignition and L = Headlight

| L | I | D | X |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | $I L \bar{I} D$ |
| 1 | 0 | 0 | $I L \bar{I} \bar{D}$ |
| 1 | 0 | 1 | $I L \bar{I} D$ |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | $I L I D$ |

$X = \bar{L} I D + L \bar{I} \bar{D} + L \bar{I} D + L I D$

$X = ID(\bar{L} + L) + L\bar{I}(\bar{D} + D)$

$X = ID + L\bar{I}$




3. A *BCD counter* produces a four-bit output representing the BCD code for the number of pulses that have been applied to the counter input. For example, after four pulses have occurred, the counter outputs are $DCBA = 0100_2 = 4_{10}$. The counter resets to 0000 on the tenth pulse and starts counting over again. In other words, the *DCBA* outputs will never represent a number greater than $1001_2 = 9_{10}$.
   i. Design the logic circuit that produces a HIGH output whenever the count is 2, 3, or 9. Use K mapping and take advantage of the don't-care conditions.

| D | C | B | A | X |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | x |
| 1 | 0 | 1 | 1 | x |
| 1 | 1 | 0 | 0 | x |
| 1 | 1 | 0 | 1 | x |
| 1 | 1 | 1 | 0 | x |
| 1 | 1 | 1 | 1 | x |

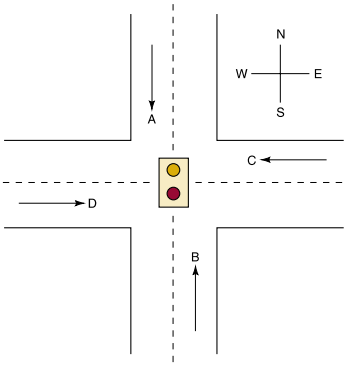
ii. Repeat for $x$= 1 when $DCBA$ = 3,4,5,8.

| D | C | B | A | X |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | x |
| 1 | 0 | 1 | 1 | x |
| 1 | 1 | 0 | 0 | x |
| 1 | 1 | 0 | 1 | x |
| 1 | 1 | 1 | 0 | x |
| 1 | 1 | 1 | 1 | x |

4. The figure below shows the intersection of a main highway with a secondary access road. Vehicle-detection sensors are placed along lanes *C* and *D* (main road) and lanes *A* and *B* (access road). These sensor outputs are LOW (0) when no vehicle is present and HIGH (1) when a vehicle is present. The intersection traffic light is to be controlled according to the following logic:

i. The north-south (N-S) traffic light will be green whenever *both* lanes *C* and *D* are occupied.
ii. The N-S light will be green whenever *either C* or *D* is occupied but lanes *A* and *B* are not *both* occupied.
iii. The East-West (E-W) light will be green whenever *both* lanes *A* and *B* are occupied but *C* and *D* are not *both* occupied.
iv. The E-W light will also be green when *either A* or *B* is occupied while *C* and *D* are *both* vacant.
v. The N-S light will be green when *no* vehicles are present.

Design a logic circuit to control the traffic light using the sensor outputs *A*, *B*, *C,* and *D* as inputs. There should be two outputs, N-S and E-W, that go HIGH when the corresponding light is to be *green.* Simplify the circuit as much as possible and show *all* steps.




**Solution**

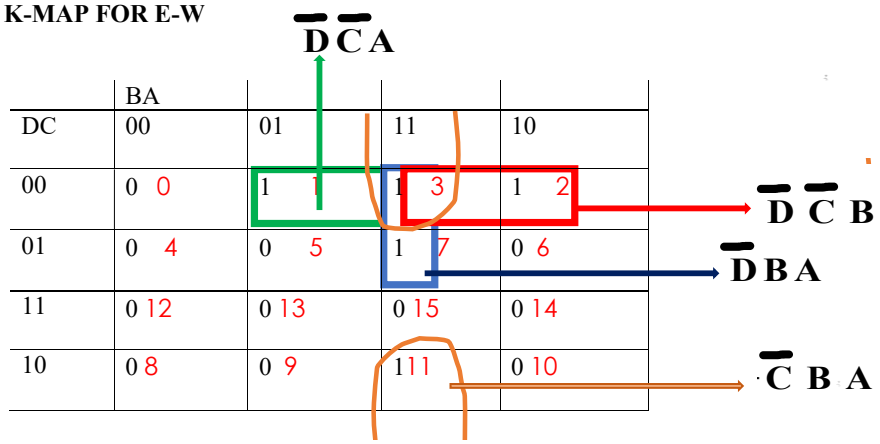
**Truth Table**

| | INPUTS | | | | OUTPUTS | |
|---|---|---|---|---|---|---|
| | **D** | **C** | **B** | **A** | **N-S** | **E-W** |
| 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 | 1 |
| 2 | 0 | 0 | 1 | 0 | 0 | 1 |
| 3 | 0 | 0 | 1 | 1 | 0 | 1 |
| 4 | 0 | 1 | 0 | 0 | 1 | 0 |
| 5 | 0 | 1 | 0 | 1 | 1 | 0 |
| 6 | 0 | 1 | 1 | 0 | 1 | 0 |

| | | | | | | |
|---|---|---|---|---|---|---|
| 7 | 0 | 1 | 1 | 1 | 0 | 1 |
| 8 | 1 | 0 | 0 | 0 | 1 | 0 |
| 9 | 1 | 0 | 0 | 1 | 1 | 0 |
| 10 | 1 | 0 | 1 | 0 | 1 | 0 |
| 11 | 1 | 0 | 1 | 1 | 0 | 1 |
| 12 | 1 | 1 | 0 | 0 | 1 | 0 |
| 13 | 1 | 1 | 0 | 1 | 1 | 0 |
| 14 | 1 | 1 | 1 | 0 | 1 | 0 |
| 15 | 1 | 1 | 1 | 1 | 1 | 0 |

From the truth table, it is obvious that N-S = NOT E-W ($\overline{\text{E-W}}$)

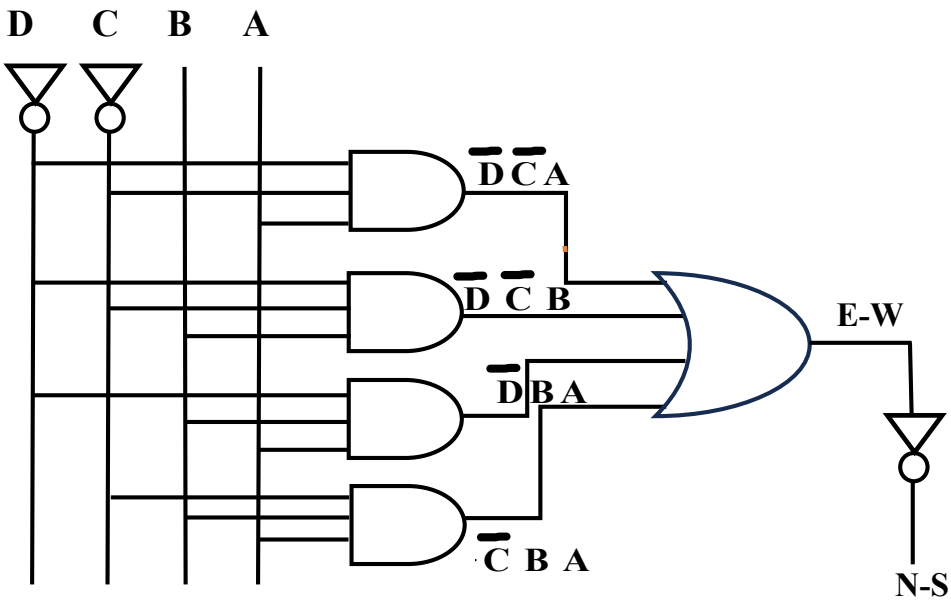
**K-MAP FOR E-W**




$$\text{E-W} = \overline{D}\,\overline{C}\,A + \overline{D}\,\overline{C}\,B + \overline{D}\,B\,A + \overline{C}\,B\,A$$

$$\text{N-S} = \overline{\text{E-W}}$$

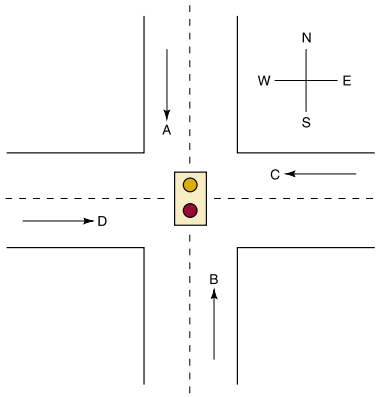
$$\text{N-S} = \overline{\overline{D}\,\overline{C}\,A + \overline{D}\,\overline{C}\,B + \overline{D}\,B\,A + \overline{C}\,B\,A}$$




5. The figure below shows the intersection of a main highway with a secondary access road. Vehicle-detection sensors are placed along lanes *C* and *D* (main road) and lanes *A* and *B* (access road). These sensor outputs are LOW (0) when no vehicle is present and HIGH (1) when a vehicle is present. The intersection traffic light is to be controlled according to the following logic:

i. The east-west (E-W) traffic light will be green whenever *both* lanes *C* and *D* are occupied.
ii. The E-W light will be green whenever *either C* or *D* is occupied but lanes *A* and *B* are not *both* occupied.
iii. The north-south (N-S) light will be green whenever *both* lanes *A* and *B* are occupied but *C* and *D* are not *both* occupied.
iv. The N-S light will also be green when *either A* or *B* is occupied while *C* and *D* are *both* vacant.
v. The E-W light will be green when *no* vehicles are present.

Design a logic circuit to control the traffic light using the sensor outputs *A*, *B*, *C,* and *D* as inputs. There should be two outputs, N-S and E-W, that go HIGH when the corresponding light is to be *green.* Simplify the circuit as much as possible and show *all* steps.



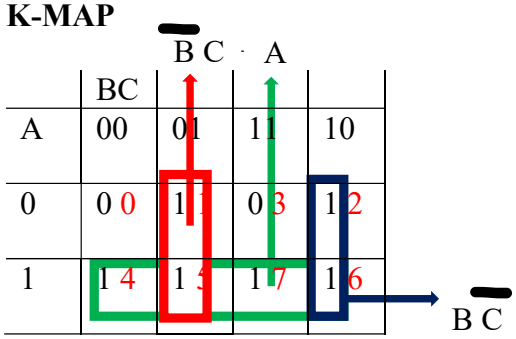

| D | C | B | A | E/W | N/S | |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 0 | |
| 0 | 0 | 0 | 1 | 1 | 0 | |
| 0 | 0 | 1 | 0 | 1 | 0 | |
| 0 | 0 | 1 | 1 | 1 | 0 | |
| 0 | 1 | 0 | 0 | 0 | 1 | $\overline{A}\,B\,\overline{C}\,\overline{D}$ |
| 0 | 1 | 0 | 1 | 1 | 0 | |
| 0 | 1 | 1 | 0 | 1 | 0 | |
| 0 | 1 | 1 | 1 | 1 | 0 | |
| 1 | 0 | 0 | 0 | 0 | 1 | $A\,\overline{B}\,\overline{C}\,\overline{D}$ |
| 1 | 0 | 0 | 1 | 1 | 0 | |
| 1 | 0 | 1 | 0 | 1 | 0 | |
| 1 | 0 | 1 | 1 | 1 | 0 | |
| 1 | 1 | 0 | 0 | 0 | 1 | $A\,B\,\overline{C}\,\overline{D}$ |
| 1 | 1 | 0 | 1 | 0 | 1 | $A\,B\,\overline{C}\,D$ |
| 1 | 1 | 1 | 0 | 0 | 1 | $A\,B\,C\,\overline{D}$ |
| 1 | 1 | 1 | 1 | 1 | 0 | |

5. Design a logic circuit that controls the passage of signal *A* according to the following requirements:

a. Output *X* will equal *A* when control inputs *B* and *C* are the same.
b. *X* will remain HIGH when *B* and *C* are different.

| | A | B | C | X |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 |
| 2 | 0 | 1 | 0 | 1 |
| 3 | 0 | 1 | 1 | 0 |
| 4 | 1 | 0 | 0 | 1 |
| 5 | 1 | 0 | 1 | 1 |
| 6 | 1 | 1 | 0 | 1 |
| 7 | 1 | 1 | 1 | 1 |

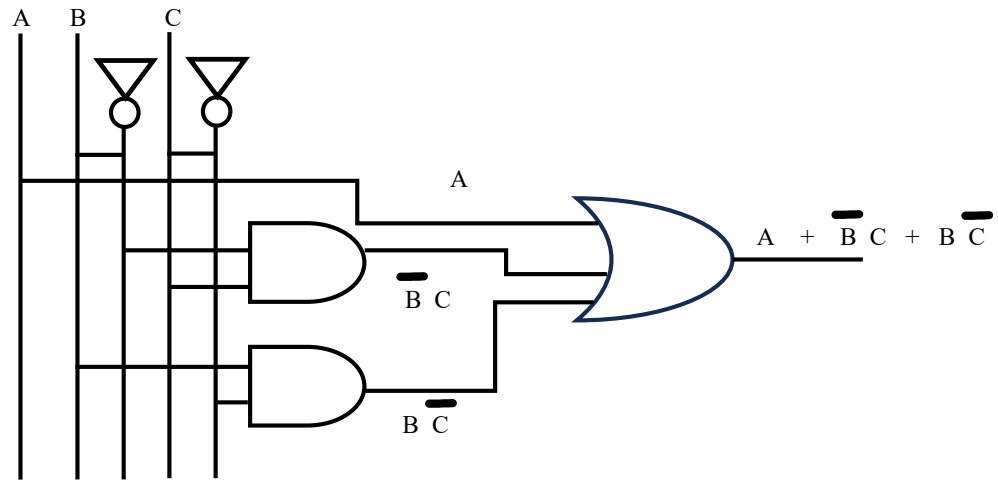
**K-MAP**



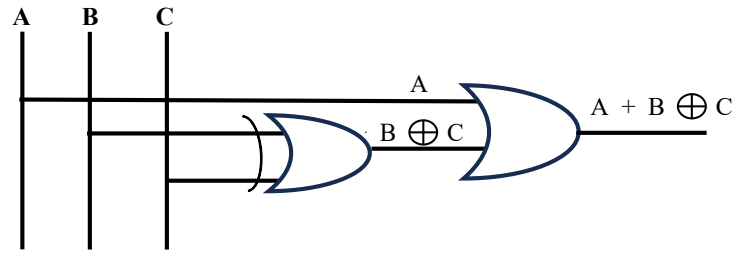

$X = A + \overline{B}\,C + B\,\overline{C}$

OR

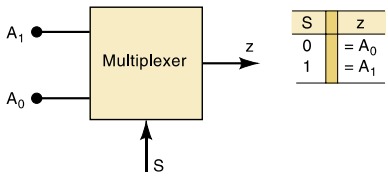
$X = A + B \oplus C$

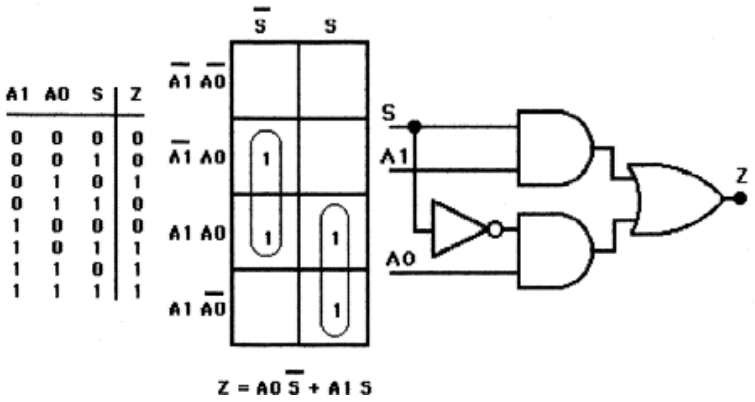



***OR***

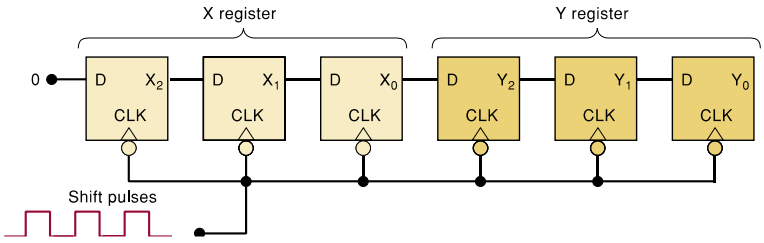



Design a logic circuit that has two signal inputs, $A_1$ and $A_0$, and a control input $S$ so that it functions according to the requirements given in Figure below






7 The figure below shows two three-bit shift registers connected so that the contents of the *X* register will be serially transferred (shifted) into register *Y*. D flip-flops are used for each shift register because this requires fewer connections than J-K flip-flops. Notice how $X_0$, the last FF of register *X*, is connected to the *D* input of $Y_2$, the first FF of register *Y*. Thus, as the shift pulses are applied, the information transfer takes place as follows: $X_2 : X_1 : X_0 : Y_2 : Y_1 : Y_0$.The $X_2$ FF will go to a state determined by its *D* input. For now, *D* will be held LOW, so that $X_2$ will go LOW on the first pulse and will remain there.




Before any shift pulses are applied, the contents of the X register are 101 (i.e.,$X_2 = 1, X_1 = 0, X_0 = 1$) and the register is at 000.

a. What will be the contents of each FF after the occurrence of the third shift pulse?

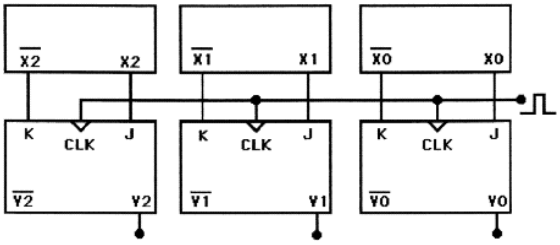
| $X_2$ | $X_1$ | $X_0$ | $Y_2$ | $Y_1$ | $Y_0$ | |
|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 0 | 0 | Initial state |
| 0 | 1 | 0 | 1 | 0 | 0 | After first Pulse |
| 0 | 0 | 1 | 0 | 1 | 0 | After second Pulse |
| 0 | 0 | 0 | 1 | 0 | 1 | After third Pulse |

b. What will be the contents of each FF after the occurrence of the sixth shift pulse?

| $X_2$ | $X_1$ | $X_0$ | $Y_2$ | $Y_1$ | $Y_0$ | |
|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 0 | 0 | Initial state |
| 0 | 1 | 0 | 1 | 0 | 0 | After the first Pulse |
| 0 | 0 | 1 | 0 | 1 | 0 | After the second Pulse |
| 0 | 0 | 0 | 1 | 0 | 1 | After the third Pulse |
| 1 | 0 | 0 | 0 | 1 | 0 | After the fourth Pulse |
| 0 | 1 | 0 | 0 | 0 | 1 | After the fifth Pulse |
| 1 | 0 | 1 | 0 | 0 | 0 | After the sixth Pulse |

At the end of the sixth pulse, all the FFs are in the initial state

**Classwork**




Refer to the previous figure. Assume that the initial contents of the registers are $X_2 = 0, X_1 = 1, X_0 = 0, Y_2 = 1, Y_1 = 1, Y_0 = 0$. Also, assume that the $D$ input of $X_2$ is held HIGH. Determine the value of each FF output after the occurrence of the fourth shift pulse.

8. Consider a counter circuit that contains six FFs wired in the arrangement (i.e., $Q_5$, $Q_4$, $Q_3$, $Q_2$, $Q_1$, $Q_0$).

(a) Determine the counter's MOD number. MOD number = $2^6 = 64$.

(b) Determine the frequency at the output of the last FF ($Q_5$) when the input clock frequency is 1 MHz.

The frequency at the last FF will equal the input clock frequency divided by the MOD number. That is,

$f$(at $Q_5$) = 1MHz/64 = 15.625 kHz

(c) What is the range of counting states for this counter?

The counter will count from $000000_2$ to $111111_2$ (0 to $63_{10}$) for a total of 64 states. Note that the number of states is the same as the MOD number.

(d) Assume a starting state (count) of 000000. What will be the counter's state after 129 pulses?

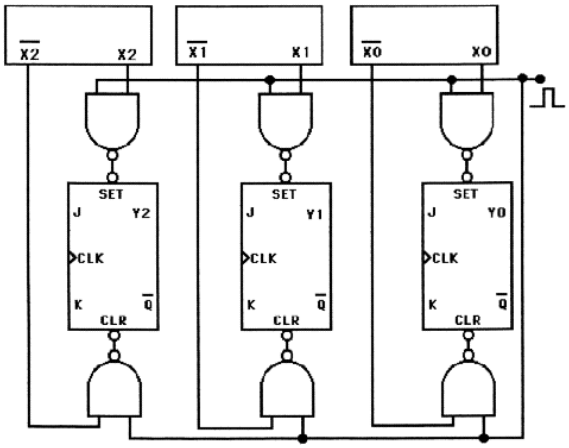
Because this is a MOD-64 counter, every 64 clock pulses will bring the counter back to its starting state. Therefore, after 128 pulses, the count is back to 000000. The 129th pulse brings the counter to the 000001 counter.

e. How many FFs are required for a counter that will count 0 to $255_{10}$?

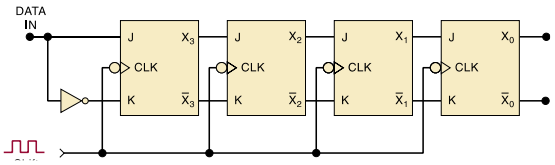
f. What is the MOD number of the counter in question 2?

**9.** (a) Draw a circuit diagram for the synchronous parallel transfer of data from one three-bit register to another using J-K flip-flops.

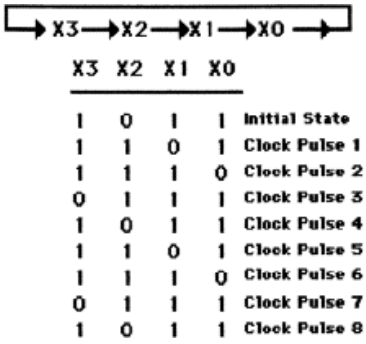
(b) Repeat for asynchronous parallel transfer.




**9**. A *recirculating* shift register is a shift register that keeps the binary information circulating through the register as clock pulses are applied. The shift register of below can be made into a circulating register by connecting $X_0$ to the DATA IN line. No external inputs are used. Assume that this circulating register starts out with 1011 stored in it (i.e.,$X_3 = 1$,$X_2 = 0$,$X_1 = 1$, and $X_0 = 1$). List the sequence of states that the register FFs go through as eight shift pulses are applied.

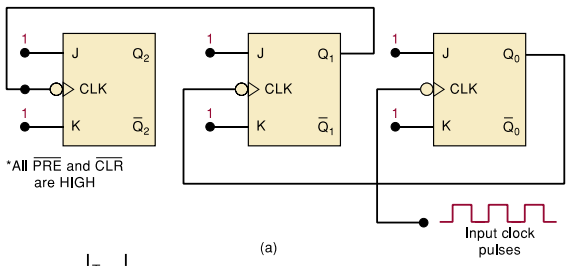



In this arrangement, the data shifts accordingly:




| X3 | X2 | X1 | X0 | |
|---|---|---|---|---|
| 1 | 0 | 1 | 1 | Initial State |
| 1 | 1 | 0 | 1 | Clock Pulse 1 |
| 1 | 1 | 1 | 0 | Clock Pulse 2 |
| 0 | 1 | 1 | 1 | Clock Pulse 3 |
| 1 | 0 | 1 | 1 | Clock Pulse 4 |
| 1 | 1 | 0 | 1 | Clock Pulse 5 |
| 1 | 1 | 1 | 0 | Clock Pulse 6 |
| 0 | 1 | 1 | 1 | Clock Pulse 7 |
| 1 | 0 | 1 | 1 | Clock Pulse 8 |

**10.** Refer to the counter circuit below and answer the following:




(a)*If the counter starts at 000, what will be the count after 13 clock pulses? After 99 pulses? After 256 pulses? This is a counter that will recycle every 8 pulses (MOD 8 counter).

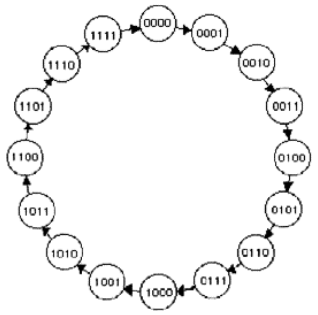
Count after 13 clock pulses is 5 (101); Count after 99 clock pulses is 3 (011); Count after 256 clock pulses is 0 (000).

(b) If the counter starts at 100, what will be the count after 13 pulses? After 99 pulses? After 256 pulses?

Count after 13 clock pulses is 1 (001); Count after 99 clock pulses is 7 (111); Count after 256 clock pulses is 4 (100).

(c) Connect a fourth J-K FF ($X_3$) to this counter and draw the state transition diagram for this 4-bit counter.

State diagram for a MOD-16 counter




11. A photodetector circuit is being used to generate a pulse each time a customer walks into a certain establishment. The pulses are fed to an eight-bit counter. The counter is used to count these pulses as a means for determining how many customers have entered the store. After closing the store, the proprietor checks the counter and finds that it shows a count of $00001001_2 = 9_{10}$. He knows that this is incorrect because there were many more than nine people in his store. Assuming that the counter circuit is working properly, what could be the reason for the discrepancy? Proposed an ideal counter for this store and justify your reason.

The counter can count up to $2^8$=256 customers, after which it resets back to 00000000. It will be ideal to have a counter that can count up $2^9$= 512 customers or $2^{10}$= 1024 customers if the shop proprietor can justify that the customers that patronise his store daily are very close to that, and if he can afford the cost of either of these counters.

12. Four large tanks at a chemical plant contain different liquids being heated. Liquid-level sensors are being used to detect whenever the level in tank *A* or tank *B* rises above a predetermined level. Temperature sensors in tanks *C* and *D* detect when the temperature in either of these tanks drops below a prescribed temperature limit. Assume that the liquid-level sensor outputs *A* and *B* are LOW when the level is satisfactory and HIGH when the level is too high. Also, the temperature-sensor outputs *C* and *D* are LOW when the temperature is satisfactory and HIGH when the temperature is too low. Design a logic circuit that will detect whenever the level in tank *A* or tank *B* is too high at the same time that the temperature in either tank *C* or tank *D* is too low.

**Design condition**
To design a logic circuit that activates an alarm when **either** the liquid level in **Tank A or B** is too high **and** the temperature in **Tank C or D** is too low, we can follow these steps:

**Sensor Output Interpretation**

- **Liquid-Level Sensors (A and B):**
  - **LOW (0):** Level is satisfactory.
  - **HIGH (1):** Level is too high.
- **Temperature Sensors (C and D):**

- **LOW (0):** Temperature is satisfactory.
- **HIGH (1):** Temperature is too low.

**Logic Design**
We aim to trigger an alarm when:

- **(A OR B) = 1** (i.e., either Tank A or B has a high liquid level) **AND**
- **(C OR D) = 1** (i.e., either Tank C or D has a low temperature)

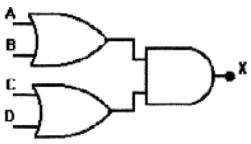
**Boolean Expression:**
**Alarm = (A + B) · (C + D)**
Where:

- + denotes the **OR** operation.
- · denotes the **AND** operation.

**Logic Circuit Implementation**
To implement this logic, we'll use basic logic gates:

1. **OR Gates:**
   - **OR1:** Inputs A and B → Output X
   - **OR2:** Inputs C and D → Output Y
2. **AND Gate:**
   - Inputs X and Y → Output Alarm

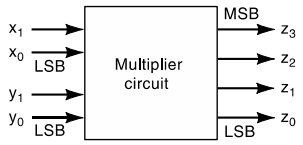
**Circuit Diagram:**




Truth Table

| | A (Tank A High) | B (Tank B High) | C (Tank C Low Temp) | D (Tank D Low Temp) | Alarm |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 2 | 0 | 0 | 1 | 0 | 0 |
| 3 | 0 | 0 | 1 | 1 | 0 |
| 4 | 0 | 1 | 0 | 0 | 0 |
| 5 | 0 | 1 | 0 | 1 | 1 |
| 6 | 0 | 1 | 1 | 0 | 1 |
| 7 | 0 | 1 | 1 | 1 | 1 |
| 8 | 1 | 0 | 0 | 0 | 0 |
| 9 | 1 | 0 | 0 | 1 | 1 |
| 10 | 1 | 0 | 1 | 0 | 1 |
| 11 | 1 | 0 | 1 | 1 | 1 |
| 12 | 1 | 1 | 0 | 0 | 0 |
| 13 | 1 | 1 | 0 | 1 | 1 |
| 14 | 1 | 1 | 1 | 0 | 1 |
| 15 | 1 | 1 | 1 | 1 | 1 |

13. The figure below represents a multiplier circuit that takes two-bit binary numbers, $x_1x_0$ and $y_1y_0$, and produces an output binary number $z_3z_2z_1z_0$ that is equal to the arithmetic product of the two input numbers. Design the logic circuit for the multiplier. (*Hint:* The logic circuit will have four inputs and four outputs.)




| INPUTS | | | | OUTPUTS | | | |
|---|---|---|---|---|---|---|---|
| Y1 | Y0 | X1 | X0 | Z3 | Z2 | Z1 | Z0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 | 0 | 0 | 1 |
| 0 | 1 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 1 | 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 0 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 |

Four outputs are required since the largest product will be 1001(nine) for the case where Y1Y0=11(three), and X1X0=11(three). Z0 is the LSB of the output.

Since there are four separate outputs, then four seperate circuits must be designed; one for each output.

**Output $Z_3$:** $Z_3 = 1$ only for a single case in the T-T. Thus, $\mathbf{Z_3 = Y_1\, Y_0\, X_1\, X_0}$

**Output $Z_2$:** $Z_2$ is HIGH for three cases. Thus, $Z_2 = Y_1\overline{Y_0}X_1X_0 + Y_1Y_0X_1\overline{X_0}$ $\overline{Z2} = Y_1X_1(\overline{Y_0 + X_0}) = Y_1X_1\,(\overline{Y_0}\,\overline{X_0})$

**Output Z1:** Z1 is HIGH for six cases. Thus,

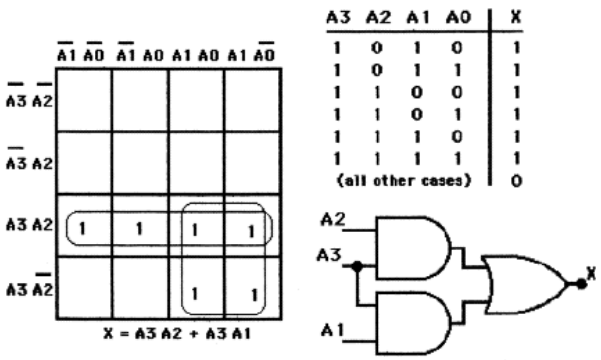
$$Z1 = Y1\overline{Y0}X1\overline{X0} + Y1Y0X1\overline{X0} + \overline{Y1}\,\overline{Y0}X1\overline{X0} + Y1\overline{Y0}X1X0 + \overline{Y1}Y0X1X0 + Y1Y0X1X0$$

$$Z1 = Y0X1(\overline{Y1 + X0}) + Y1\overline{X0}(\overline{Y0 + X1})$$

**Output Z0:** Z0 is HIGH for four cases.

$Z0 = Y1\overline{Y0}X1\overline{X0} + Y1Y0X1X0 + \overline{Y1}Y0X1X0 + Y1Y0X1X0$ . Thus, $Z0 = Y0X0$

4.27 A BCD code is being transmitted to a remote receiver. The bits are $A3$, $A2$, $A1$, and $A0$, with $A3$ as the MSB. The receiver circuitry includes a *BCD error detector* circuit that examines the received code to see if it is a legal BCD code (i.e., ≤1001). Design this circuit to produce a HIGH for any error condition.

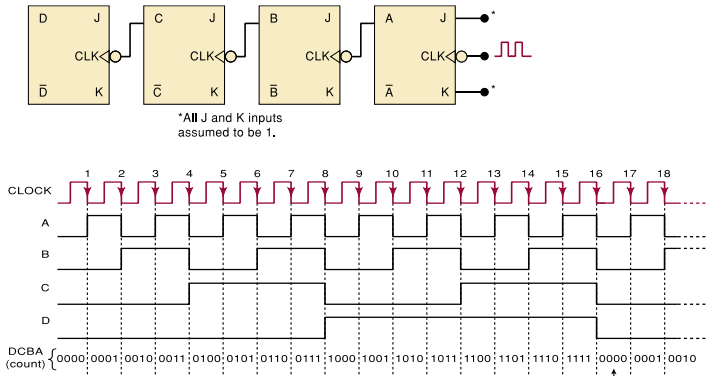



| A3 | A2 | A1 | A0 | X |
|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |
| (all other cases) | | | | 0 |

## ASYNCHRONOUS (RIPPLE) COUNTERS

The figure below shows a four-bit binary counter circuit such as the one discussed in Chapter 5. Recall the following points concerning its operation:

1. The clock pulses are applied only to the *CLK* input of flip-flop *A*. Thus, flip-flop *A* will toggle (change to its opposite state) each time the clock pulses make a negative (HIGH-to-LOW) transition. Note that $J$ $K$ 1 for all FFs.
2. The normal output of flip-flop *A* acts as the *CLK* input for flip-flop *B*, and so flip-flop *B* will toggle each time the *A* output goes from 1 to 0. Similarly, flip-flop *C* will toggle when *B* goes from 1 to 0, and flip-flop *D* will toggle when *C* goes from 1 to 0.
3. FF outputs *D*, *C*, *B*, and *A* represent a four-bit binary number, with *D* as the MSB. Let's assume that all FFs have been cleared to the 0 state (CLEAR inputs are not shown). The waveforms in Figure 7-1 show that a binary counting sequence from 0000 to 1111 is followed as clock pulses are continuously applied.
4. After the Negative Going Transition (NGT) of the fifteenth clock pulse has occurred, the counter FFs are in the 1111 condition. On the sixteenth NGT, flip-flop *A* goes from 1 to 0, which causes flip-flop *B* to go from 1 to 0, and so on, until the
5. counter is in the 0000 state. In other words, the counter has gone through one complete cycle (0000 through 1111) and has *recycled* back to 0000. From this point, it will begin a new counting cycle as subsequent clock pulses are applied.
6. In this counter, each FF output drives the *CLK* input of the next FF. This type of counter arrangement is called an **asynchronous counter** because the FFs do not change states in exact synchronism with the applied clock pulses; only flip-flop *A* responds to the clock pulses. FF *B* must wait for FF *A* to change states before it can toggle; FF *C* must wait for FF *B*, and so on. Thus, there is a delay between the responses of successive FFs. This delay is typically 5–20 ns per FF. In some cases, as we shall see, this delay can be troublesome. This type of counter is also often referred to as a **ripple counter** because of the way the FFs respond one after another in a kind of rippling effect. We will use the terms *asynchronous counter* and *ripple counter* interchangeably.




### MOD Number

The counter in the above Figure has 16 distinctly different states (0000 through 1111). Thus, it is a *MOD-16 ripple counter.* Recall that the **MOD number** is generally equal to the number of states that the counter goes through in each complete cycle before it recycles back to its starting state. The MOD number can be increased simply by adding more FFs to the counter. That is,

$$\text{MOD number} = 2^N$$

where $N$ is the number of FFs connected in the arrangement of the figure above.

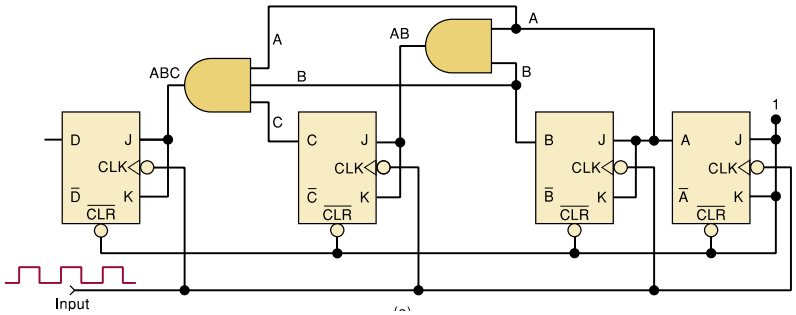
**Drawbacks of Ripple counter**

Ripple counters are the simplest type of binary counters because they require the fewest components to produce a given counting operation. They do, however, have one major drawback, which is caused by their basic principle of operation: each FF is triggered by the transition at the output of the preceding FF. Because of the inherent propagation delay time ($t_{pd}$) of each FF, this means that the second FF will not respond until a time $t_{pd}$ after the first FF receives an active clock transition; the third FF will not respond until a time equal to 2 * $t_{pd}$ after that clock transition; and so on. In other words, the propagation delays of the FFs accumulate so that the *N*th FF cannot change states until a time equal to $N$ * $t_{pd}$ after the clock transition occurs.

## SYNCHRONOUS (PARALLEL) COUNTERS

The problems encountered with ripple counters are caused by the accumulated FF propagation delays; stated another way, the FFs do not all change states simultaneously in synchronism with the input pulses. These limitations can be overcome with the use of **synchronous** or **parallel counters** in which all of the FFs are triggered simultaneously (in parallel) by the clock input pulses.

Because the input pulses are applied to all the FFs, some means must be used to control when a FF is to toggle and when it is to remain unaffected by a clock pulse. This is accomplished by using the *J* and *K* inputs and is illustrated in the Figure below for a four-bit, MOD-16 synchronous counter.




If we compare the circuit arrangement for this synchronous counter with its asynchronous counterpart, we can see the following notable differences:

- The *CLK* inputs of all of the FFs are connected together so that the input clock signal is applied to each FF simultaneously.
- Only flip-flop *A*, the LSB, has its *J* and *K* inputs permanently at the HIGH level. The *J*, *K* inputs of the other FFs are driven by some combination of FF outputs.
- The synchronous counter requires more circuitry than does the asynchronous counter.

### Circuit Operation

For this circuit to count properly, on a given NGT of the clock, only those FFs that are supposed to toggle on that NGT should have $J$ $K$ 1 when that NGT occurs. Let's look at the counting sequence in Figure 7-5(b) to see what this means for each FF.

The counting sequence shows that the *A* flip-flop must change states at each NGT. For this reason, its *J* and *K* inputs are permanently HIGH so that it will toggle on each NGT of the clock input.

The counting sequence shows that flip-flop *B* must change states on each NGT that occurs while $A = 1$. For example, when the count is 0001, the next NGT must toggle *B* to the 1 state; when the count is 0011, the next NGT must toggle *B* to the 0 state; and so on. This operation is accomplished by connecting output *A* to the *J* and *K* inputs of flip-flop *B* so that $J = K = 1$ only when $A = 1$.

The counting sequence shows that flip-flop *C* must change states on each NGT that occurs while *A*= *B*= 1. For example, when the count is 0011, the next NGT must toggle *C* to the 1 state; when the count is 0111, the next NGT must toggle *C* to the 0 state; and so on. By connecting the logic signal *AB* to FF *C*'s *J* and *K* inputs, this FF will toggle only when *A*= *B*= 1.

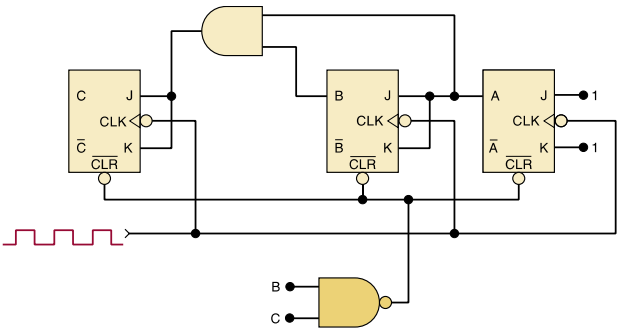
In a like manner, we can see that flip-flop *D* must toggle on each NGT that occurs while *A* = *B* = *C* = 1. When the count is 0111, the next NGT must toggle *D* to the 1 state; when the count is 1111, the next NGT must toggle *D* to the 0 state. By connecting the logic signal *ABC* to FF *D*'s *J* and *K* inputs, this FF will toggle only when *A* =*B* = *C* = 1.

The basic principle for constructing a synchronous counter can therefore be stated as follows:

**Each FF should have its *J* and *K* inputs connected so that they are HIGH only when the outputs of all lower-order FFs are in the HIGH state.**

## COUNTERS WITH MOD NUMBERS < $2^N$

The basic synchronous counter of above Figure is limited to MOD numbers that are equal to $2^N$, where *N* is the number of FFs. This value is the maximum MOD number that can be obtained using *N* flip-flops. The basic counter can be modified to produce MOD numbers less than $2^N$ by allowing the counter to *skip states* that are normally part of the counting sequence. One of the most common methods for doing this is illustrated in below, where a three-bit counter is shown. Disregarding the NAND gate for a moment, we can see that the counter is a MOD-8 binary counter that will count in sequence from 000 to 111. However, the presence of the NAND gate will alter this sequence as follows:




1. The NAND output is connected to the asynchronous CLEAR inputs of each FF. As long as the NAND output is HIGH, it will have no effect on the counter. When it goes LOW, however, it will clear all of the FFs so that the counter immediately goes to the 000 state.

2. The inputs to the NAND gate are the outputs of the *B* and *C* flip-flops, and so the NAND output will go LOW whenever *B* *C* 1. This condition will occur when the counter goes from the 101 state to the 110 state on the NGT of input pulse 6. The LOW at the NAND output will immediately (generally within a few nanoseconds) clear the counter to the 000 state. Once the FFs have been cleared, the NAND output goes back HIGH be- cause the *B* *C* 1 condition no longer exists.
3. The counting sequence is, therefore,

*CBA*

000 ←
001
010
011
100
101
110 → (temporary state needed to clear counter)

Although the counter does go to the 110 state, it remains there for only a few nanoseconds before it recycles to 000. Thus, we can essentially say that this counter counts from 000 (zero) to 101 (five) and then recycles to 000. It essentially skips 110 and 111 so that it goes through only six different states; thus, it is a MOD-6 counter.

## Displaying Counter States

To have a visible display of how a counter is changing states in response to the input pulses. LED (Light Emitting Diode) is used. When turned ON, it is HIGH and vice versa. Also, a seven-segment display can be used.

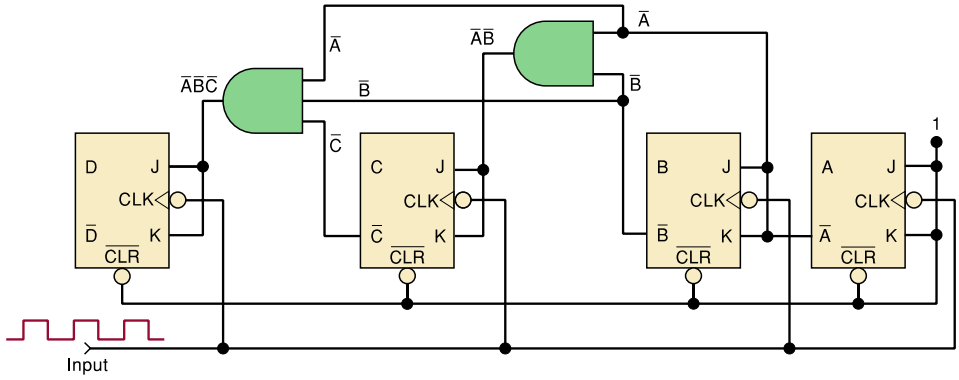
## Decade Counters/BCD Counters

The MOD-10 counter is also referred to as a **decade counter**. A decade counter is any counter that has 10 distinct states, no matter what the sequence. A decade counter which counts in sequence from 0000 (zero) through 1001 (decimal 9), is also commonly called a **BCD counter** because it uses only the 10 BCD code groups 0000, 0001, . . . , 1000, and 1001. To reiterate, any MOD-10 counter is a decade counter; and any decade counter that counts in binary from 0000 to 1001 is a BCD counter.

Decade counters, especially the BCD type, find widespread use in applications where pulses or events are to be counted and the results displayed on some type of decimal numerical readout.

### SYNCHRONOUS DOWN

We have seen that using the output of lower-order FFs to control the toggling of each FF creates a synchronous **up counter**. A synchronous **down counter** is constructed similarly except that we use the inverted FF outputs to control the higher-order *J*, *K* inputs.




Draw the circuit diagram for a MOD-32 synchronous counter.

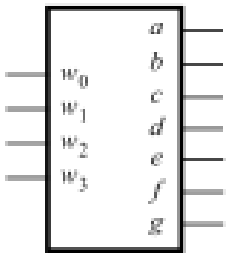
Draw the circuit diagram for a MOD-64 synchronous counter.

Draw a synchronous, MOD-32, down counter.

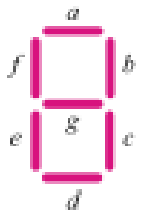
**BCD to 7-segment display**

**Code converters**

One common example is a BCD-to-7-segment decoder, which converts one binary-coded decimal (BCD) digit into information suitable for driving a digit-oriented display. As illustrated in Figure *A* below, the circuit converts the BCD digit into seven signals that are used to drive the segments in the display. Each segment is a small light-emitting diode (LED), which glows when driven by an electrical signal. The segments are labelled from *a* to *g* in the figure. The truth table for the BCD-to-7-segment decoder is given in Figure *c*. For each valuation of the inputs *w*3, . . . , *w*0, the seven outputs are set to display the appropriate BCD digit. Note that the last 6 rows of a complete 16-row truth table are not shown. They represent don't-care conditions because they are not legal BCD codes and will never occur in a circuit that deals with BCD data.




(a) Code converter




(b) 7-segment display

| $w_3$ | $w_2$ | $w_1$ | $w_0$ | $a$ | $b$ | $c$ | $d$ | $e$ | $f$ | $g$ |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |

State machines